Building an autonomous AI insurance quoting system to outperform Handle AI

The full technical stack for a WhatsApp-native, autonomous multi-carrier insurance quoting platform targeting the Mexican market is achievable today by combining AI browser automation (Skyvern + Browser Use), stealth Playwright for portal scraping, serverless job orchestration (QStash/BullMQ), real-time SSE streaming, and encrypted credential management. Handle AI — backed by a16z with $6M seed — automates quoting for 75+ Mexican brokers theaiinsider but suffers from portal brittleness, no public API integrations, and no self-serve model. The architecture below addresses every weakness while delivering a system that is WhatsApp-first, fully autonomous, and dramatically cheaper to operate.

---

1. AI browser automation: Skyvern SDK and Browser Use

Two frameworks dominate AI-powered browser automation in 2026: **Skyvern** (computer-vision-based, $0.05/step) and **Browser Use** (DOM-parsing-based, MIT-licensed, 83.5k GitHub stars). Both can navigate insurance portals without hardcoded selectors, but they differ fundamentally in approach and cost structure.

Skyvern SDK (v1.0.24) — production code

Skyvern uses screenshots and vision models to understand pages visually, making it resilient to portal layout changes. GitHub GitHub The Python SDK is fully async:

```python
```

```python
from skyvern import Skyvern

client = Skyvern(api_key=os.getenv("SKYVERN_API_KEY"))

result = await client.run_task(
    prompt="Log into the Qualitas agent portal at agentes.qualitas.com.mx. Navigate
    url="https://agentes.qualitas.com.mx",
    engine="skyvern-2.0",
    data_extraction_schema={
        "type": "object",
        "properties": {
            "premium_annual": {"type": "string", "description": "Annual premium in M
            "premium_monthly": {"type": "string", "description": "Monthly premium in
            "coverage_type": {"type": "string"},
            "deductible": {"type": "string"},
            "policy_number": {"type": "string"}
        }
    },
    totp_identifier="qualitas_agent_totp",
    webhook_url="https://your-app.vercel.app/api/webhooks/skyvern",
    proxy_location="RESIDENTIAL",
    max_steps=30
)
```

Key Skyvern capabilities for insurance automation include **browser sessions** for maintaining authenticated state across multiple tasks, Skyvern skyvern **browser profiles** that persist cookies and tokens indefinitely, Skyvern and **workflow blocks** for multi-step automations with conditional logic, loops, and file downloads. Skyvern skyvern Downloaded PDFs are accessible via `result.downloaded_files` — each entry contains a pre-signed URL and checksum. Skyvern

Browser Use (v0.12.5) — faster, cheaper alternative

Browser Use parses DOM via Chrome DevTools Protocol rather than screenshots, GitHub making it **3-5x faster** and cheaper per action. GitHub PyPI It scores **89% on WebVoyager** Keywords AI versus Skyvern's 85.8%: Skyvern

```python
```

```python
from browser_use import Agent, Browser, ChatBrowserUse
from pydantic import BaseModel
from typing import List

class InsuranceQuote(BaseModel):
    carrier: str
    premium_annual: float
    premium_monthly: float
    coverage: str
    deductible: float

class QuoteResults(BaseModel):
    quotes: List[InsuranceQuote]

agent = Agent(
    task="""Go to https://agentes.qualitas.com.mx. Login with provided credentials.
    Navigate to Cotizador Express. Quote auto insurance for:
    - Vehicle: 2024 Toyota Camry LE (AMIS clave 0426103)
    - Zip: 06600, Male, Age 35
    - Coverage: Amplia, Deductible: $5,000 MXN
    Extract all premium and coverage details.""",
    llm=ChatBrowserUse(),
    browser=Browser(use_cloud=True),  # Stealth browser with anti-detection
    sensitive_data={
        "username": agent_credentials["username"],
        "password": agent_credentials["password"]
    },
    output_model_schema=QuoteResults,
)
history = await agent.run(max_steps=50)
```

The optimal strategy is a hybrid approach: use Browser Use for fast, cost-effective quoting on portals with stable DOM structures, and fall back to Skyvern for portals with aggressive bot detection or frequent layout changes. Browser Use's **MIT license** OpenAlternative GitHub makes it ideal for a commercial product, while Skyvern's **AGPL-3.0** OpenAlternative Skypage requires careful licensing consideration.

---

2. Playwright stealth mode for direct portal automation

For maximum control and speed, direct Playwright automation with stealth plugins bypasses the LLM overhead entirely. This is the best approach for portals with known, stable structures.

Anti-detection configuration

```python
from playwright.async_api import async_playwright
from playwright_stealth import Stealth
import random

async with Stealth().use_async(async_playwright()) as p:
    browser = await p.chromium.launch(
        headless=True,
        args=['--disable-blink-features=AutomationControlled', '--no-sandbox']
    )
    context = await browser.new_context(
        user_agent='Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 Chr
        viewport={'width': 1920, 'height': 1080},
        locale='es-MX',
        timezone_id='America/Mexico_City',
    )
    # Manual fingerprint hardening
    await context.add_init_script("""
        Object.defineProperty(navigator, 'webdriver', { get: () => undefined });
        Object.defineProperty(navigator, 'plugins', { get: () => [1, 2, 3, 4, 5] });
        const getParameter = WebGLRenderingContext.prototype.getParameter;
        WebGLRenderingContext.prototype.getParameter = function(param) {
            if (param === 37445) return 'Intel Inc.';
            if (param === 37446) return 'Intel Iris OpenGL Engine';
            return getParameter.call(this, param);
        };
        window.chrome = { runtime: {} };
    """)
```

Cascading dropdowns — critical for AMIS vehicle selection

Mexican insurance portals use cascading dropdowns (marca → modelo → año → versión) that load options via AJAX. This pattern handles them reliably:

```javascript
```

```
async function fillCascadingDropdowns(page, selections) {
  for (const { selector, value, apiPattern } of selections) {
    await page.waitForSelector(`${selector}:not([disabled])`, { timeout: 10000 });
    if (apiPattern) {
      const responsePromise = page.waitForResponse(
        resp => resp.url().includes(apiPattern) && resp.status() === 200,
        { timeout: 15000 }
      );
      await page.selectOption(selector, { label: value });
      await responsePromise;
    } else {
      await page.selectOption(selector, { label: value });
    }
    await page.waitForTimeout(500);
  }
}

// Insurance vehicle form with AMIS catalog navigation
await fillCascadingDropdowns(page, [
  { selector: '#marca', value: 'Toyota', apiPattern: '/api/modelos' },
  { selector: '#modelo', value: 'Camry', apiPattern: '/api/anios' },
  { selector: '#anio', value: '2024', apiPattern: '/api/versiones' },
  { selector: '#version', value: 'LE 2.5L Aut', apiPattern: null },
]);
```

CAPTCHA integration with 2Captcha

```python
from twocaptcha import TwoCaptcha
solver = TwoCaptcha(os.environ['CAPTCHA_API_KEY'])

site_key = await page.evaluate(
    'document.querySelector("[data-sitekey]")?.getAttribute("data-sitekey")'
)
if site_key:
    result = solver.recaptcha(sitekey=site_key, url=page.url)
    await page.evaluate(f'document.querySelector("#g-recaptcha-response").value="{re
    await page.click('#login-button')
```

PDF download handling

```javascript
const downloadPromise = page.waitForEvent('download');
await page.click('#download-policy-pdf');
const download = await downloadPromise;
await download.saveAs(`./downloads/${download.suggestedFilename()}`);
```

---

3. Next.js 15 SSE for real-time progressive quote delivery

Server-Sent Events deliver individual carrier quotes to the UI as they arrive Pedro Alonso — users see results progressively rather than waiting for all carriers to respond. This is a massive UX advantage over Handle AI's batch approach.

SSE route handler

```typescript
```

```
// app/api/sse/quotes/route.ts
import { NextRequest } from 'next/server';

export const dynamic = 'force-dynamic';
export const runtime = 'nodejs';

export async function GET(req: NextRequest) {
  const encoder = new TextEncoder();
  const requestId = req.nextUrl.searchParams.get('requestId');

  const stream = new ReadableStream({
    start(controller) {
      function send(event: string, data: unknown) {
        controller.enqueue(encoder.encode(`event: ${event}\ndata: ${JSON.stringify(d
      }

      send('connected', { requestId, timestamp: Date.now() });

      // Subscribe to quote results from job queue (via Redis pub/sub or polling)
      const pollInterval = setInterval(async () => {
        const results = await getNewQuoteResults(requestId);
        for (const quote of results) {
          send('quote_result', quote);
        }
        const status = await getRequestStatus(requestId);
        send('progress', { percent: status.percent, message: status.message });
        if (status.complete) {
          send('complete', { totalQuotes: status.totalQuotes, bestPrice: status.best
          clearInterval(pollInterval);
          clearInterval(heartbeat);
          setTimeout(() => controller.close(), 100);
        }
      }, 1000);

      const heartbeat = setInterval(() => {
        try { controller.enqueue(encoder.encode(`: heartbeat\n\n`)); } catch { clear
      }, 15000);

      req.signal.addEventListener('abort', () => {
        clearInterval(pollInterval);
        clearInterval(heartbeat);
        controller.close();
      });
```

```
    },
  });

  return new Response(stream, {
    headers: {
      'Content-Type': 'text/event-stream',
      'Cache-Control': 'no-cache, no-transform',
      'Connection': 'keep-alive',
      'X-Accel-Buffering': 'no',
      'Content-Encoding': 'none',
    },
  });
}
```

React client hook with reconnection

```typescript
```

```typescript
// hooks/useQuoteStream.ts
'use client';
import { useState, useEffect, useRef, useCallback } from 'react';

export function useQuoteStream(requestId: string | null) {
  const [quotes, setQuotes] = useState<Quote[]>([]);
  const [progress, setProgress] = useState(0);
  const [status, setStatus] = useState<'idle' | 'streaming' | 'complete' | 'error'>(
  const esRef = useRef<EventSource | null>(null);

  useEffect(() => {
    if (!requestId) return;
    setStatus('streaming');
    const es = new EventSource(`/api/sse/quotes?requestId=${requestId}`);
    esRef.current = es;

    es.addEventListener('quote_result', (e) => {
      setQuotes(prev => [...prev, JSON.parse(e.data)]);
    });
    es.addEventListener('progress', (e) => setProgress(JSON.parse(e.data).percent));
    es.addEventListener('complete', () => { setStatus('complete'); es.close(); });
    es.onerror = () => { if (es.readyState === EventSource.CLOSED) setStatus('error'

    return () => es.close();
  }, [requestId]);

  return { quotes, progress, status };
}
```

Critical gotcha: never `await` long operations inside `start()` — Next.js buffers the entire response. Use the background IIFE pattern shown above, or a `TransformStream`.

---

4. WhatsApp Cloud API for insurance quote delivery

WhatsApp is the dominant communication channel for Mexican insurance brokers. The Cloud API (v25.0) supports interactive messages perfect for quote comparisons.

Formatted quote comparison message

```typescript
```

```
function formatQuoteComparison(quotes: InsuranceQuote[]): string {
  let msg = `📊 *Comparación de Cotizaciones*\n━━━━━━━━━━━━━━━━━━━━━━\n\n`;
  for (const q of quotes) {
    msg += `🏷️ *${q.carrier}*\n`;
    msg += `┌──────────────────────\n`;
    msg += `│ 💰 Prima anual: *$${q.premium_annual} MXN*\n`;
    msg += `│ 📋 Deducible: $${q.deductible} MXN\n`;
    msg += `│ 🏥 Cobertura: ${q.coverage_type}\n`;
    msg += `│ ⭐ Calificación: ${q.rating}\n`;
    msg += `└──────────────────────\n\n`;
  }
  msg += `_Toca un botón para seleccionar tu plan_`;
  return msg;
}
```

Interactive reply buttons for quote selection

typescript

```
async function sendQuoteWithButtons(to: string, quote: FormattedQuote) {
  await axios.post(`https://graph.facebook.com/v25.0/${PHONE_NUMBER_ID}/messages`, {
    messaging_product: 'whatsapp',
    to,
    type: 'interactive',
    interactive: {
      type: 'button',
      header: { type: 'text', text: 'Tu Cotización de Seguro' },
      body: { text: formatQuoteComparison(quote.quotes) },
      footer: { text: 'Cotización válida por 30 días' },
      action: {
        buttons: [
          { type: 'reply', reply: { id: 'accept_quote', title: '✅ Aceptar' } },
          { type: 'reply', reply: { id: 'modify_quote', title: '✏️ Modificar' } },
          { type: 'reply', reply: { id: 'compare_more', title: '🔄 Más opciones' } }
        ],
      },
    },
  }, { headers: { Authorization: `Bearer ${ACCESS_TOKEN}` } });
}
```

List messages for plan browsing (max 10 rows across max 10 sections)

```json
{
  "interactive": {
    "type": "list",
    "body": { "text": "Selecciona una aseguradora para ver detalles:" },
    "action": {
      "button": "Ver Opciones",
      "sections": [{
        "title": "Mejores Precios",
        "rows": [
          { "id": "qualitas_quote", "title": "Quálitas", "description": "$8,500/año
          { "id": "gnp_quote", "title": "GNP Seguros", "description": "$9,200/año —
          { "id": "axa_quote", "title": "AXA", "description": "$8,900/año — Individu
        ]
      }]
    }
  }
}
```

PDF quote delivery via media upload

```typescript
```

```
async function sendQuotePDF(to: string, pdfPath: string) {
  const form = new FormData();
  form.append('file', fs.createReadStream(pdfPath));
  form.append('type', 'application/pdf');
  form.append('messaging_product', 'whatsapp');

  const { data: media } = await axios.post(
    `https://graph.facebook.com/v25.0/${PHONE_NUMBER_ID}/media`, form,
    { headers: { ...form.getHeaders(), Authorization: `Bearer ${ACCESS_TOKEN}` } }
  );

  await axios.post(`https://graph.facebook.com/v25.0/${PHONE_NUMBER_ID}/messages`, {
    messaging_product: 'whatsapp', to, type: 'document',
    document: { id: media.id, filename: 'Cotizacion_Seguro.pdf', caption: 'Tu cotiza
  }, { headers: { Authorization: `Bearer ${ACCESS_TOKEN}` } });
}
```

Template messages with **UTILITY** category enable proactive outreach outside the 24-hour window — critical for sending quote expiration reminders and renewal notifications. Infobip Templates support up to **10 buttons** (mix of quick_reply, URL, phone_number, and copy_code types). Infobip

---

5. Supabase Vault for encrypted credential storage

Supabase Vault uses **pgsodium** with **XChaCha20-Poly1305 AEAD** encryption. Secrets are encrypted at rest in `vault.secrets` and decrypted on-the-fly through the `vault.decrypted_secrets` view — decrypted values never touch disk. GitHub

Complete credential management system

```sql
```

```sql
-- Metadata table with RLS for multi-tenant isolation
CREATE TABLE public.portal_credentials (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  agent_id UUID NOT NULL REFERENCES auth.users(id),
  portal_name TEXT NOT NULL,
  portal_url TEXT,
  username_secret_id UUID NOT NULL,
  password_secret_id UUID NOT NULL,
  is_active BOOLEAN DEFAULT true,
  created_at TIMESTAMPTZ DEFAULT now(),
  UNIQUE(agent_id, portal_name)
);

ALTER TABLE public.portal_credentials ENABLE ROW LEVEL SECURITY;
CREATE POLICY "agent_isolation" ON public.portal_credentials
  FOR ALL USING (agent_id = auth.uid()) WITH CHECK (agent_id = auth.uid());

-- Security-definer function to store credentials
CREATE OR REPLACE FUNCTION store_portal_credential(
  p_portal_name TEXT, p_portal_url TEXT, p_username TEXT, p_password TEXT
) RETURNS UUID LANGUAGE plpgsql SECURITY DEFINER SET search_path = public AS $$
DECLARE v_agent_id UUID; v_username_sid UUID; v_password_sid UUID; v_cred_id UUID;
BEGIN
  v_agent_id := auth.uid();
  IF v_agent_id IS NULL THEN RAISE EXCEPTION 'Not authenticated'; END IF;
  v_username_sid := vault.create_secret(p_username, 'portal_' || v_agent_id || '_' |
  v_password_sid := vault.create_secret(p_password, 'portal_' || v_agent_id || '_' |
  INSERT INTO portal_credentials (agent_id, portal_name, portal_url, username_secret
  VALUES (v_agent_id, p_portal_name, p_portal_url, v_username_sid, v_password_sid) R
  RETURN v_cred_id;
END; $$;

-- Retrieve decrypted credentials (server-side only)
CREATE OR REPLACE FUNCTION get_portal_credential(p_portal_name TEXT)
RETURNS TABLE (portal_name TEXT, username TEXT, password TEXT)
LANGUAGE plpgsql SECURITY DEFINER SET search_path = public AS $$
DECLARE v_cred RECORD;
BEGIN
  SELECT pc.username_secret_id, pc.password_secret_id INTO v_cred
  FROM portal_credentials pc WHERE pc.agent_id = auth.uid() AND pc.portal_name = p_p
  IF NOT FOUND THEN RETURN; END IF;
  RETURN QUERY SELECT p_portal_name,
    (SELECT ds.decrypted_secret FROM vault.decrypted_secrets ds WHERE ds.id = v_cred
```

```
    (SELECT ds.decrypted_secret FROM vault.decrypted_secrets ds WHERE ds.id = v_cred
END; $$;
```

Important: pgsodium is pending deprecation as a standalone extension, but Supabase has confirmed **Vault's API will remain unchanged**. Always access Vault through security-definer functions — never expose `vault.decrypted_secrets` to API roles. GitHub

---

6. AES-256-GCM encryption service for Node.js

For application-layer encryption of credentials before they even reach Supabase, this service provides an additional encryption layer with key rotation support:

```typescript
```

```
import { randomBytes, scryptSync, createCipheriv, createDecipheriv } from 'node:cryp

const ALGORITHM = 'aes-256-gcm';
const IV_LENGTH = 12;  // NIST-recommended for GCM
const AUTH_TAG_LENGTH = 16;
const SALT_LENGTH = 16;
const KEY_LENGTH = 32;

export class CredentialEncryptionService {
  private keys: Map<number, string>;
  private currentVersion: number;

  constructor(passwords: Record<number, string>) {
    this.keys = new Map(Object.entries(passwords).map(([v, p]) => [Number(v), p]));
    this.currentVersion = Math.max(...this.keys.keys());
  }

  encrypt(plaintext: string): string {
    const salt = randomBytes(SALT_LENGTH);
    const key = scryptSync(this.keys.get(this.currentVersion)!, salt, KEY_LENGTH, {
    const iv = randomBytes(IV_LENGTH);
    const cipher = createCipheriv(ALGORITHM, key, iv, { authTagLength: AUTH_TAG_LENG
    const ciphertext = Buffer.concat([cipher.update(plaintext, 'utf8'), cipher.final
    const authTag = cipher.getAuthTag();
    // Pack: [version:1B][salt:16B][iv:12B][authTag:16B][ciphertext:NB]
    const versionBuf = Buffer.alloc(1);
    versionBuf.writeUInt8(this.currentVersion);
    return Buffer.concat([versionBuf, salt, iv, authTag, ciphertext]).toString('base
  }

  decrypt(payload: string): string {
    const buf = Buffer.from(payload, 'base64');
    const version = buf.readUInt8(0);
    const salt = buf.subarray(1, 17);
    const iv = buf.subarray(17, 29);
    const authTag = buf.subarray(29, 45);
    const ciphertext = buf.subarray(45);
    const key = scryptSync(this.keys.get(version)!, salt, KEY_LENGTH, { N: 16384, r:
    const decipher = createDecipheriv(ALGORITHM, key, iv, { authTagLength: AUTH_TAG_
    decipher.setAuthTag(authTag);
    return Buffer.concat([decipher.update(ciphertext), decipher.final()]).toString('
  }
```

```
  addKey(version: number, password: string) { this.keys.set(version, password); this
  reEncrypt(payload: string): string { return this.encrypt(this.decrypt(payload)); }
  needsReEncryption(payload: string): boolean {
    return Buffer.from(payload, 'base64').readUInt8(0) < this.currentVersion;
  }
}
```

The **12-byte IV** is critical — GCM is designed for 96-bit IVs per NIST SP 800-38D. CopyProgramming Using 16 bytes introduces an unnecessary hashing step and marginally reduces security. The version byte in the wire format enables seamless algorithm migration. Medium

---

7. Job orchestration: BullMQ fan-out and QStash for serverless

BullMQ FlowProducer for parallel carrier quoting

```typescript
```

```
import { Queue, FlowProducer, Worker } from 'bullmq';

const connection = {
  host: process.env.UPSTASH_REDIS_ENDPOINT!,
  port: 6379,
  password: process.env.UPSTASH_REDIS_PASSWORD!,
  maxRetriesPerRequest: null,  // Required by BullMQ
  tls: {},                      // Required by Upstash
};

const flowProducer = new FlowProducer({ connection });

// Fan-out: one quote request → parallel carrier jobs
async function createQuoteFanOut(requestId: string, vehicleData: VehicleData) {
  const carriers = ['qualitas', 'gnp', 'axa', 'hdi', 'chubb'];
  return flowProducer.add({
    name: 'aggregate-quotes',
    queueName: 'quote-requests',
    data: { requestId, vehicleData },
    children: carriers.map(carrier => ({
      name: `quote-${carrier}`,
      queueName: 'carrier-quotes',
      data: { requestId, carrier, vehicleData },
      opts: { attempts: 3, backoff: { type: 'exponential', delay: 2000 } },
    })),
  });
}

// Aggregation worker — runs after ALL children complete
const aggregationWorker = new Worker('quote-requests', async (job) => {
  const childrenValues = await job.getChildrenValues();
  const quotes = Object.values(childrenValues);
  return { requestId: job.data.requestId, quotes, bestPrice: Math.min(...quotes.map(
}, { connection, concurrency: 3 });
```

Critical serverless limitation: BullMQ workers need persistent processes — they cannot run inside Vercel Functions. Deploy workers on **Railway or Fly.io**, and use Vercel Functions only as producers (`Queue.add()`). Upstash recommends a **Fixed plan** for BullMQ to avoid per-request billing costs from background polling.

QStash for pure serverless architecture

QStash is the better fit for a fully Vercel-native deployment — it **pushes** messages to HTTP endpoints rather than requiring polling workers: Upstash

```typescript
import { Client } from '@upstash/qstash';
const qstash = new Client({ token: process.env.QSTASH_TOKEN! });

// Fan-out quoting to carrier endpoints
async function dispatchCarrierQuotes(requestId: string, carriers: string[]) {
  return Promise.all(carriers.map(carrier =>
    qstash.publishJSON({
      url: `${process.env.APP_URL}/api/process-carrier-quote`,
      body: { requestId, carrier },
      retries: 3,
      callback: `${process.env.APP_URL}/api/callbacks/quote-complete`,
      failureCallback: `${process.env.APP_URL}/api/callbacks/quote-failed`,
      deduplicationId: `quote-${requestId}-${carrier}`,
    })
  ));
}
```

```typescript
// app/api/process-carrier-quote/route.ts
import { verifySignatureAppRouter } from '@upstash/qstash/nextjs';

async function handler(req: Request) {
  const { requestId, carrier } = await req.json();
  const quote = await runPortalAutomation(carrier, requestId);
  return Response.json({ success: true, quote });
}

export const POST = verifySignatureAppRouter(handler); // Signature verification is
```

Vercel Cron Jobs for scheduled tasks

```json
```

```
{
  "crons": [
    { "path": "/api/cron/sync-amis-catalog", "schedule": "0 5 * * *" },
    { "path": "/api/cron/check-portal-health", "schedule": "*/15 * * * *" },
    { "path": "/api/cron/renewal-reminders", "schedule": "0 9 * * 1-5" }
  ]
}
```

Hobby plans are limited to **once-per-day** cron jobs. Pro plans support more frequent schedules. All schedules run in **UTC** — no timezone configuration available.

---

8. Circuit breaker pattern with distributed Redis state

Insurance portals are unreliable. A circuit breaker prevents cascading failures when a carrier portal goes down, using Redis-backed Lua scripts for atomic state transitions across all serverless instances:

```typescript
```

```
export class RedisCircuitBreaker extends EventEmitter {
  private readonly redis: Redis;
  private readonly serviceId: string;
  private readonly config: { failureThreshold: number; successThreshold: number; tim

  async fire<T>(fn: () => Promise<T>): Promise<T> {
    const now = Date.now();
    // Atomic pre-call check via Lua script
    const [action, state, , , , retryAfter] = await this.redis.eval(LUA_PRE_CALL, 1,
      now, this.config.timeout, this.config.halfOpenMaxAttempts) as string[];

    if (action === 'REJECT_OPEN') throw new CircuitOpenError(this.serviceId, parseIn
    if (action === 'REJECT_HALF_OPEN') throw new CircuitHalfOpenLimitError(this.serv

    try {
      const result = await fn();
      await this.redis.eval(LUA_RECORD_SUCCESS, 1, `cb:${this.serviceId}`, Date.now(
      return result;
    } catch (error) {
      await this.redis.eval(LUA_RECORD_FAILURE, 1, `cb:${this.serviceId}`, Date.now(
      throw error;
    }
  }
}

// Usage: wrap each carrier portal call
const qualitasBreaker = new RedisCircuitBreaker({
  redis, serviceId: 'qualitas-portal',
  failureThreshold: 5, successThreshold: 2, timeout: 60_000, halfOpenMaxAttempts: 3,
});

const quote = await qualitasBreaker.fire(() => scrapeQualitasQuote(vehicleData));
```

Three Lua scripts handle **pre-call checks**, **success recording**, and **failure recording** atomically. The state machine transitions: **CLOSED** (normal) → **OPEN** (after `failureThreshold` failures, reject all calls) → **HALF_OPEN** (after `timeout` ms, allow `halfOpenMaxAttempts` probe calls) → back to CLOSED or OPEN depending on probe results.

9. Mexican insurance portals and the AMIS vehicle catalog

Portal landscape

All five major Mexican carriers use web-based agent portals with standard username/password authentication tied to the agent's **CNSF cédula** (insurance regulatory license number):

Carrier	Portal URL	Technology	Key Features
Quálitas	`agentes.qualitas.com.mx`	Java/Struts (.do endpoints)	Cotizador Express — issue in <5 min
GNP Seguros	`intermediarios.gnp.com.mx`	Google App Engine	CRM-style dashboard, Ideas GNP
AXA Mexico	`distribuidores.axa.com.mx`	Modern SPA	Tokenizer (MITEC), 8,000+ agents
HDI Seguros	`portalagentes.hdi.com.mx`	JavaScript SPA	CODA virtual assistant, iDriving telematics
Chubb Mexico	`chubb.com/mx-es` → ChubbNet	Legacy ABA + modern Chubb	Multiple portals (agents, auto dealers)

No public APIs exist for any of these carriers — portal automation via browser automation is the only viable integration method, which is exactly how Handle AI operates. **No widespread CAPTCHA or MFA was detected** on agent portals, making automation more feasible. The standard quoting flow across all carriers follows the same pattern: login → select marca/modelo/año/versión (from AMIS catalog) → enter driver data → select coverage → get premium.

AMIS vehicle catalog

The **AMIS vehicle catalog** (Catálogo de Claves Vehiculares) is the universal vehicle identification standard used by all **83+ AMIS-member** insurance companies. Each vehicle is assigned a **7-character alphanumeric clave**: positions 1-2 identify the manufacturer (e.g., `04` = Nissan), positions 3-4 the model, and positions 5-7 the trim/version.

There is no official AMIS API. The catalog is distributed as:

- **DOF Anexo 15**: Published annually in the Diario Oficial de la Federación (freely downloadable)

- **AMIS InformaWeb**: Legacy Tomcat 4.1.29 system at `amis.com.mx/InformaWeb/` (restricted access)
- **AMDA annual updates**: Latest version (December 2024) covers 2025 model year vehicles

The practical approach: **parse the DOF PDF publication annually**, load it into a Supabase database table, and expose it as a searchable API for the quoting system. Each carrier's portal has the AMIS catalog embedded in their dropdown flows — your system should maintain its own copy to pre-populate vehicle selection and reduce automation steps.

An emerging option is **MexInsurance (mexinsurance.net)**, a new API-native platform (currently in beta) that promises REST API access for quoting with Mexican carriers — worth monitoring as a potential supplement to portal automation.

---

10. How this system beats Handle AI

Handle AI raised **$6M from Andreessen Horowitz** (March 2026) and serves **75+ Mexican brokers** with four AI agents: quoting, collections, reconciliation, and renewals. Founded by Alfonso de los Ríos (former CEO of Nowports, a $1.1B logistics startup), Handle claims **94% faster** claims registration and **8 hours/week** recovered per executive.

Handle AI's exploitable weaknesses

Portal brittleness: Handle relies exclusively on browser automation without fallback strategies. A dual approach — API-first where available (MexInsurance beta) plus AI browser automation with circuit breakers — would be dramatically more reliable.

No self-serve model: Handle requires sales demos and custom onboarding ("operating in less than a week"). A WhatsApp-first self-onboarding flow where agents simply message the bot, authenticate their portal credentials, and start quoting within minutes would be a 100x faster onboarding experience.

No public pricing: Handle's enterprise pricing likely excludes small independent agents. A usage-based model ($0.50-$2 per multi-carrier quote) with a generous free tier would capture the long tail of Mexico's thousands of small insurance agents.

Mexico-only: Handle has no US presence. Building with English/Spanish bilingual support and US carrier portal integrations from day one creates a much larger addressable market.

No AMS integration: Handle doesn't integrate with major agency management systems. Even in Mexico, CRM integration (Salesforce, HubSpot, Dynamics — all mentioned on Handle's own site) could be a differentiator if done deeper.

Architecture advantage

The system described in this report is fundamentally more advanced than Handle AI in several ways: **progressive SSE streaming** shows quotes as they arrive rather than waiting for all carriers; **circuit breakers** gracefully handle portal outages; **hybrid AI automation** (Browser Use + Skyvern + direct Playwright) provides multiple fallback layers; **encrypted credential Vault** with application-layer AES-256-GCM provides defense-in-depth; and **QStash fan-out** enables truly serverless scaling without persistent infrastructure costs.

The total infrastructure cost for this stack — Vercel Pro ($20/mo), Upstash Redis ($10/mo), Supabase Pro ($25/mo), QStash ($1/100K messages), Skyvern ($0.05/step for fallback) — runs under **$100/month** at moderate volume, compared to Handle's likely enterprise pricing.

Conclusion: the key architectural decisions

The most important architectural insight across this research is the **layered automation strategy**: use direct Playwright with stealth plugins for portals with known, stable structures (fastest, cheapest); fall back to Browser Use with LLM-driven DOM parsing when portal layouts change (flexible, still fast); and escalate to Skyvern's computer vision approach only for portals with aggressive bot detection (most resilient, most expensive). Each layer is wrapped in a Redis-backed circuit breaker that automatically routes around failures.

The **QStash fan-out pattern** is the clear winner over BullMQ for a Vercel-native deployment — it eliminates the need for persistent worker infrastructure entirely. Combined with **SSE progressive delivery**, users see the first carrier quote within seconds while remaining carriers continue processing in parallel.

For the Mexican market specifically, the **AMIS vehicle catalog** must be maintained as a local database (parsed annually from DOF publications), and the five carrier portals (Quálitas, GNP, AXA, HDI, Chubb) share enough structural similarity in their quoting flows that a single template with per-carrier configuration can handle them all. The absence of widespread CAPTCHA and MFA on Mexican agent portals is a significant tailwind for automation — stealth Playwright with Mexican residential proxies should achieve reliable access without needing CAPTCHA solving services for most portals.

The competitive window against Handle AI is now — they're early-stage with only 75 clients and no self-serve model. A WhatsApp-native, self-onboarding system that charges per quote rather than per seat could capture the Mexican independent agent market before Handle AI scales.